African eDevelopment Resource Centre

ASP.NET Core 8 Training

5 days

Presentation

With .NET Core: Learn the latest cross-platform Web technology from Microsoft!
Our ASP.NET Core 8 training course will introduce you to Microsoft's open-source, cross-platform technology. You'll be able to create modern applications connected to the cloud and the Internet. You'll be able to deploy them locally or in the cloud.
The aim of this course is to introduce you to the concepts of website development and cross-platform application development using .NET Core 8, and to show you all the new features.
At the end of this course, you'll be able to start developing a new ASP.NET MVC Core application with confidence, and migrate an existing application to ASP.NET MVC Core.
In this training, as in all our training courses, we'll be using the latest stable version (presenting) the new features of ASP.NET Core 8 and MVC Core 3 with the new C# 12 language).

Objectives

- Learn how .NET Core works
- Discover the features of the ASP.NET Core and MVC Core frameworks
- How to write a REST service● Creating a Web application with Visual Studio
- Design an ASP.NET application using best development practices
- Testing and debugging ASP.NET Core applications
- Learn about the framework's advanced mechanisms and get started with Entity Framework Core
(EF Core)
- Be able to migrate a legacy ASP.NET MVC project to the new ASP.NET MVC Core

Target audience

Web and application developer.
Prerequisites
Knowledge of object-oriented programming, ideally in the .NET environment.
Software requirements
- Visual Studio 2019 Community (minimum)
- .NET Desktop development
- Web development and ASP.NET
- Data storage and processing

Day 1: Kick-off ASP.NET Core 3 & 5 & 6 & 7 & 8

Introduction
- Background, context and objectives
- Current .NET Framework issues
- What is ASP.NET Core?
- Advantages and disadvantages of ASP.NET Core● Constraints, when to choose ASP.NET CORE 3 vs ASP.NET Framework
- Quick overview of new features in C# 11 and operating environments
- Support & Lifecycle: LTS vs Current Lifecycle
- What's new in version 2.1 & 2.2 & 3.0 & 3.1 & 5 & 6 & the new version 8
- How do you develop in cross-platform mode?

Setting up a development environment
- Practical work: First project
- Package retrieval
- Command line creation
- Editing and debugging with VS Code
- Visual Studio project
- Project components
- A look back at some important conceptual building blocks:
- MVC (Model-View-Controller)
- NuGet packages
- Dependency injection
- Practical work: Implementing the day's concepts

Day 2: Advanced

- Overview of the development cycle for a new ASP.NET Core project
- Visual Studio gas pedals (Scaffolding, etc.)
- Non-regression testing, debugging and performance
- View, Model, Pages, Razor & Tags helpers
- Practical work
- Routes & Controllers
- How to declare your routes
- Routing templates: syntax and usage
- New area concept
- Attributes
- Query filters
- Url rewriter
- Practical work
- Back to dependency injection and managing Initialization Options
- Declaration
- Validation
- Practical work

Day 3: Advanced concepts

- Entity Framework (EF Core)
- Practical work
- Web API
- Authentication/Authorization/Identity Management
- CORS (Cross Origin Resource Sharing) security
- Recent problems caused by Google's "breaking ecolutions
- Enhanced safety
- Samesite cookie
- Practical work● Caching
- Localization
- Middleware
- Background tasks
- Practical work

Day 4 Synthesis of acquired concepts

- Use in cloud environments
- Azure
- AWS
- Overview of different systems based on ASP.NET Core
- Blazor
- Orchard Core
- Integration with other Open Source projects
- Swagger example
- A look back at the migration from Asp.Net 4 to Asp.Net Core (tools, tips & tricks)
- Practical work: Synthesis of elements learned in the form of a realistic project

Day 5 Apply styles

- CSS page layout
- Responsive design
- Bootstrap integration 4.4.1 Unit testing: best practices
- Implementing unit tests
- Exception management strategy

Access control: identity and authorization

- User Model customization with Identity
- Restrict access
- Security settings: tokens, cookies, session duration
- Use external providers: OAuth (Google, Facebook, Microsoft) & OpenID
- The new Principal and Identity classes, which are based in spirit on claims
- Authentication using JWT tokens
- Building your own API
- Securing your API
- Logging
- Resilient web application :
- Statefull

- Secure access control and status managementSignalR: Real-time web functionalities
- Create an ASP.NET Core project to host the SignalR server and client
- Create a SignalR hub
- Configuring your project to use SignalR
- Create SignalR client code
- Run a high-frequency application
- Introduction to gRPC

Deployment and integration

- Deploying on an IIS server
- Deploy with a Git service (VSFS, TFS, GitHub)
- Deploying to the Azure cloud
- Deploy using Docker
- Managing configuration transformation

Migrating legacy code

- Migrating earlier versions to ASP.NET Core 3.x
- Migrating an application to .NET Core
- Migrating from ASP.NET MVC 5 to ASP.NET MVC Core
- Migrating from ASP.NET Web API 2 to ASP.NET MVC Core

Teaching methods

Practical course: 60% Practical, 40% Theory. Training material distributed in digital format to all participants.

Organization

The course alternates theoretical inputs from the trainer supported by examples andbrainstorming sessions and group work.